UNIVERSIDADE DA AMAZÔNIA - UNAMA
Curso de Ciência da Computação
8º semestre - turno da noite

ATIVIDADE DEVOPS NA PRÁTICA

API de sorteios, pipeline e observabilidade

Integrante	**Matrícula**
Lucas Christian Gouveia Ferreira	04131650
Gustavo Henrique Bezerra Santiago	04128537
João Vitor Lobo do Nascimento	04138562
Luiz Henrique Assunção Carvalho	04134810
Eric Ric Monteiro Almeida	04113064

Disciplina: DevOps Tools
Professor: Rodrigo Silva Marques
2026

1. Introdução e conceitos fundamentais

O projeto utiliza uma API de sorteios feita em Python com FastAPI. A aplicação recebe limites numéricos, sorteia um valor dentro do intervalo e registra as requisições. A partir dela, foram reunidos testes automatizados, empacotamento em container, execução em Kubernetes e análise de logs.

DevOps e metodologias ágeis

DevOps é uma forma de trabalho que aproxima desenvolvimento e operações, combinando colaboração, automação e retorno frequente sobre o funcionamento do software. Isso complementa as metodologias ágeis: se uma equipe entrega alterações pequenas com frequência, ela também precisa testá-las e colocá-las em funcionamento de modo repetível.

Integração e entrega contínuas

Integração contínua (CI) consiste em integrar mudanças e verificá-las com testes e build automatizados. Neste projeto, cada push na branch main inicia o workflow e executa os testes. Se falharem, o job não segue para a publicação da imagem.

Continuous Delivery mantém uma versão testada pronta para ser liberada, mas a decisão de liberá-la pode ser manual. Em **Continuous Deployment**, a liberação para o ambiente de destino ocorre automaticamente após as verificações. O projeto instala automaticamente a aplicação em um cluster de demonstração; não faz liberação em produção.

Máquinas virtuais e containers

Uma máquina virtual executa um sistema operacional convidado sobre um hypervisor. Um container compartilha o kernel do host e reúne a aplicação e suas dependências. Com isso, containers costumam iniciar mais rápido e usar menos recursos, embora não substituam o isolamento de uma VM em todas as situações. Aqui, a imagem Docker leva a mesma API para o Kind e para a demonstração local de logs.

2. Aplicação e pipeline de CI

A API disponibiliza **GET /**, para nome e versão; **GET /health**, para verificar a disponibilidade; e **GET /sortear**, com os parâmetros min e max. Por exemplo, uma chamada com min=1 e max=100 retorna um número entre esses limites. Quando min é maior que max, a resposta é HTTP 400; um valor não inteiro produz HTTP 422.

O código foi colocado no GitHub junto ao Dockerfile, aos testes e ao arquivo .github/workflows/main.yml. Ao receber um push na main, o GitHub Actions instala as dependências, executa o Pytest, constrói a imagem e a publica no GitHub Container Registry (GHCR). A imagem recebe uma tag com o identificador do commit, permitindo associá-la à versão testada. A autenticação usa o GITHUB_TOKEN do próprio workflow.

Na execução registrada para o commit 5eee7bf, os quatro testes passaram. Eles cobrem os endpoints básicos, sorteios com limites válidos, parâmetros inválidos e campos essenciais do log. O job terminou com sucesso e publicou a imagem no GHCR.

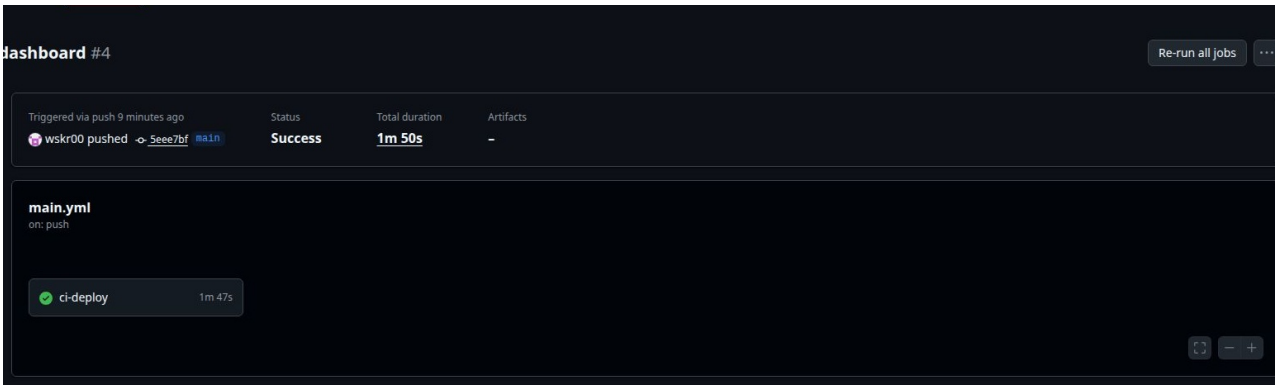


Figura 1 - Execução do workflow após push na main. Fonte: GitHub Actions, registro da execução do projeto.

A execução completa e suas etapas podem ser consultadas no link indicado ao final do relatório. O repositório é privado e exige acesso autorizado.

3. Deploy com Kubernetes e Helm

Depois da publicação, o mesmo job cria um cluster Kind temporário no runner do GitHub Actions. A imagem recém-publicada é carregada no cluster, e o Helm instala o chart do diretório helm/sorteios. O chart define um Deployment com uma réplica e um Service do tipo NodePort, além de verificações de saúde da aplicação.

Para validar a instalação, o workflow consulta o estado do Helm, lista pods e serviços, verifica o rollout e acessa a aplicação por port-forward. O registro da execução mostra o pod em estado Running, o Service NodePort e as respostas de /health e /sortear.

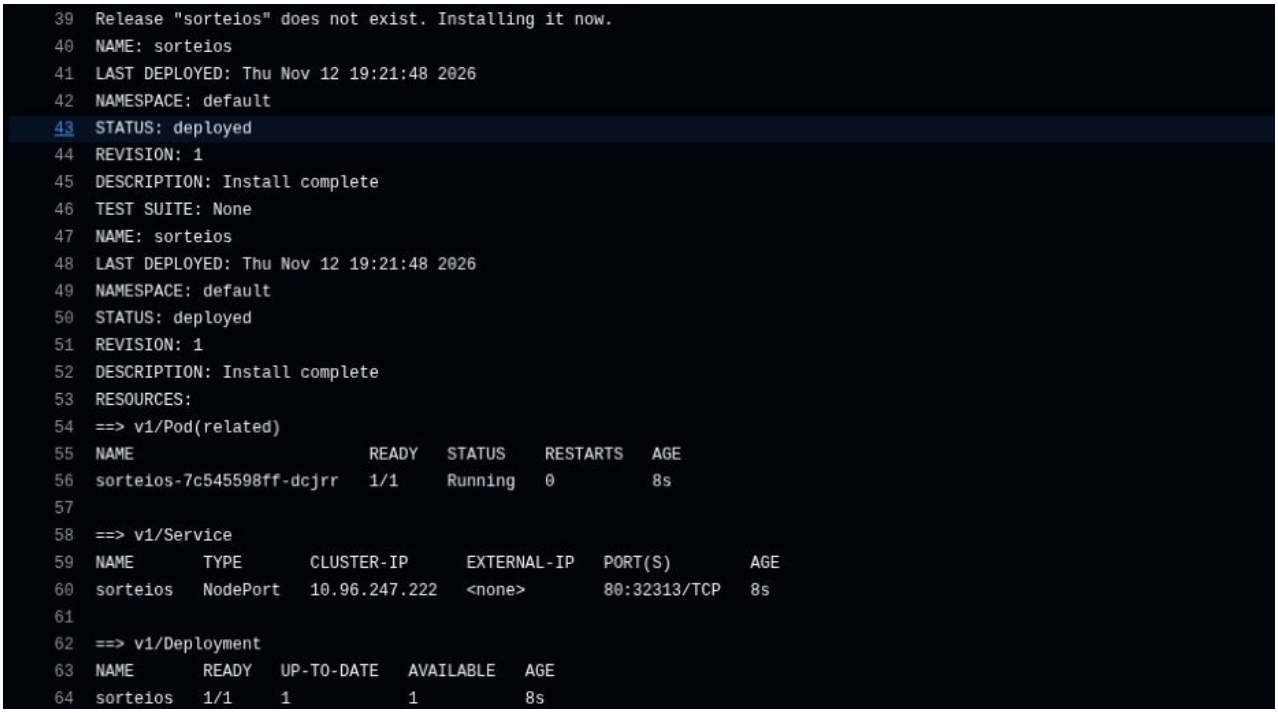


Figura 2 - Estado do release Helm, pod e Service NodePort no job do GitHub Actions. Fonte: log da execução do projeto.

Na primeira tentativa, o port-forward ainda não estava pronto e o curl registrou falha de conexão. O fluxo repetiu a tentativa e recebeu as respostas logo em seguida. O job concluiu com sucesso.

Limitação: o cluster é descartado ao fim do job. Cada push cria um novo ambiente e instala a aplicação novamente. Portanto, o resultado comprova um deploy automatizado de demonstração, sem atualizar um cluster persistente de produção.

4. Logs e métricas com Elastic Stack

Para a observabilidade, foi usada uma demonstração local independente do cluster do Actions. O Docker Compose inicia a mesma API, Elasticsearch, Kibana e Filebeat. A API grava uma linha JSON por requisição em um arquivo de um volume compartilhado, com horário, método, rota, status, duração em milissegundos e versão.

O Filebeat lê esse arquivo e envia os eventos ao Elasticsearch, que os indexa. O Kibana consulta os dados e os apresenta em um dashboard com logs recentes, contagem de requisições por status e tempo médio de resposta. Foram feitas chamadas válidas e inválidas para produzir os eventos exibidos.

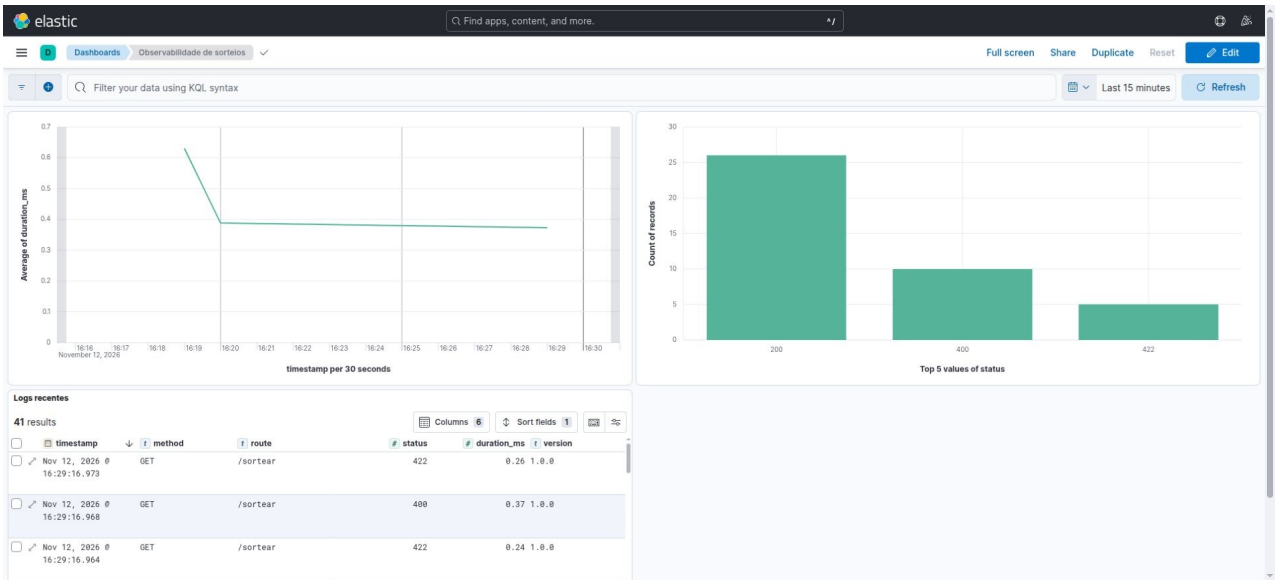


Figura 3 - Dashboard local com tempo médio, contagem por status e logs reais. Fonte: Kibana da demonstração.

Na captura, os gráficos usam o intervalo selecionado no Kibana. A tabela de logs usa uma consulta ES|QL que lista os eventos mais recentes e não depende desse mesmo seletor de tempo.

5. Dificuldades e reflexão

Dificuldades encontradas

As primeiras tentativas de publicar a imagem no GHCR terminaram com a mensagem “denied” ao enviar o manifesto. Depois, foi construída uma imagem sem a attestation automática e com metadado apontando para o repositório; a publicação passou. As execuções anteriores permanecem no histórico e não foram apresentadas como sucesso.

No Kibana 8.17.4, a busca tradicional do Discover mostrou erro ao ser incorporada ao dashboard. A tabela foi refeita com uma consulta ES|QL, que retornou os campos e registros esperados. Os testes também exibiram um aviso de depreciação de uma dependência indireta, sem impedir sua conclusão.

Reflexão crítica

A automação diminuiu a quantidade de passos manuais entre uma alteração e a verificação da aplicação: o mesmo fluxo testa, empacota e instala a versão identificada pelo commit. Os logs deram outro tipo de retorno, pois permitiram observar respostas bem-sucedidas, erros de entrada e tempos medidos em requisições reais.

Essas práticas ajudam a encontrar problemas mais cedo, mas a existência de um pipeline aprovado não garante, por si só, que uma aplicação esteja pronta para produção. Neste trabalho, o cluster é temporário e a stack Elastic é local; segurança, disponibilidade e operação de um ambiente persistente precisariam de atenção em outro contexto.

Projeto e evidências

Repositório do projeto: github.com/wskr00/atividade-devops-sorteios (privado; requer acesso).

Execução verificada: GitHub Actions - execução nº 4. As figuras foram capturadas da execução e do dashboard da atividade.

Fonte do enunciado: MARQUES, Rodrigo Silva. DevOps Tools - Atividade Avaliativa: DevOps na Prática. Material da disciplina.